

Java 8 Stream API & Collections: Coding Interview Guide

This guide contains highly optimized, production-grade Java 8 coding solutions focusing on the Stream API, lambdas, and advanced Collectors. These solutions clean up and correct standard automated OCR patterns into precise technical templates ideal for screening and technical coding interviews.

Section 1: Numerical & Array Transformations

1. Separate Odd and Even Numbers (Partitioning)

Partitions a list of integers into two groups based on a predicate, returning a map keyed by Boolean.

```
List<Integer> listOfIntegers = Arrays.asList(71, 18, 42, 21, 67, 32, 95);

Map<Boolean, List<Integer>> partitionedNumbers = listOfIntegers.stream()
    .collect(Collectors.partitioningBy(i -> i % 2 == 0));
// true -> Even numbers, false -> Odd numbers
```

2. Find Maximum and Minimum Elements

Extracts the absolute peak and valley elements from a stream via terminal comparisons.

```
Optional<Integer> max = listOfIntegers.stream().max(Comparator.naturalOrder());
Optional<Integer> min = listOfIntegers.stream().min(Comparator.naturalOrder());
```

3. Find the Second Largest Number

Sorts the sequence in reverse order and skips the primary element to capture the runner-up.

```
Optional<Integer> secondLargest = listOfIntegers.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
```

```
.skip(1)
.findFirst();
```

4. Fetch Top 3 Maximum and Minimum Numbers

Uses stream limits to truncate elements after applying sorting criteria.

```
List<Integer> max3 = listOfIntegers.stream()
    .sorted(Comparator.reverseOrder()).limit(3).collect(Collectors.toList());
```

```
List<Integer> min3 = listOfIntegers.stream()
    .sorted().limit(3).collect(Collectors.toList());
```

5. Sum of All Digits of a Number

Maps an integer into its string form, splits it into character segments, and aggregates them.

```
int num = 45632;
int sumOfDigits = Stream.of(String.valueOf(num).split(""))
    .mapToInt(Integer::parseInt)
    .sum();
```

6. Merge Two Unsorted Arrays into a Single Sorted Sequence

Concatenates low-level primitive streams before executing structural array conversions.

```
int[] a = {12, 34, 5, 7};
int[] b = {9, 2, 45, 12};

int[] sortedMerged = IntStream.concat(Arrays.stream(a), Arrays.stream(b))
    .sorted()
    .toArray();

// Variation: Merged without duplicates
int[] uniqueSortedMerged = IntStream.concat(Arrays.stream(a), Arrays.stream(b))
    .distinct()
    .sorted()
    .toArray();
```

7. Common Elements Between Two Separate Arrays

Filters elements of one array against the inclusion criteria of another array list layout.

```
List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);  
List<Integer> list2 = Arrays.asList(4, 5, 6, 7, 8);
```

```
List<Integer> common = list1.stream()  
    .filter(list2::contains)  
    .distinct()  
    .collect(Collectors.toList());
```

8. Sum of First 10 Natural Numbers & Array Aggregations

```
int sumOfTen = IntStream.range(1, 11).sum();  
  
int[] inputArray = {4, 15, 8, 22, 9};  
int arraySum = Arrays.stream(inputArray).sum();  
double arrayAvg = Arrays.stream(inputArray).average().orElse(0.0);
```

9. Reverse an Integer Array

```
int[] array = {1, 2, 3, 4, 5};  
int[] reversed = IntStream.rangeClosed(1, array.length)  
    .map(i -> array[array.length - i])  
    .toArray();
```

10. Generate Fibonacci Series

Generates an infinite sequence using an iterative seed state array, bounded using a stream limit.

```
List<Integer> fibonacci = Stream.iterate(new int[]{0, 1}, f -> new int[]{f[1], f[0] + f[1]})  
    .limit(10)  
    .map(f -> f[0])  
    .collect(Collectors.toList());
```

Section 2: Text Processing & String Manipulation

11. Frequency of Each Character in a String

Converts a string into an `IntStream` of character codes, boxes them to objects, and classifies their count parameters.

```
String inputString = "accodigital";
Map<Character, Long> charFreq = inputString.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

12. Verify Anagram Status via Streams

Compares two strings by sorting their normalized character streams and verifying sequence equality.

```
String s1 = "Anagram";
String s2 = "Nagaram";

boolean areAnagrams = Stream.of(s1.toLowerCase().split(""))
    .sorted().collect(Collectors.toList())
    .equals(Stream.of(s2.toLowerCase().split("")).sorted().collect(Collectors.toList()));
```

13. Reverse Each Word of a String

Splits text into words, applies string builder reversals on each chunk, and joins them back together using space delimiters.

```
String original = "Java Backend Developer";
String reversedWords = Arrays.stream(original.split(" "))
    .map(word -> new StringBuilder(word).reverse().toString())
    .collect(Collectors.joining(" "));
```

14. Sort List of Strings by Length Hierarchy

```
List<String> strings = Arrays.asList("Apple", "Go", "Java", "Spring", "Kubernetes");
List<String> sortedByLength = strings.stream()
    .sorted(Comparator.comparingInt(String::length))
    .collect(Collectors.toList());
```

15. Find Strings Which Start with a Number Digit

```
List<String> mixList = Arrays.asList("365Days", "Java", "100Days", "Cloud");
List<String> startsWithDigit = mixList.stream()
    .filter(s -> !s.isEmpty() && Character.isDigit(s.charAt(0)))
    .collect(Collectors.toList());
```

16. Check for Palindrome Configuration

```
String testStr = "radar";
boolean isPalindrome = IntStream.range(0, testStr.length() / 2)
    .noneMatch(i -> testStr.charAt(i) != testStr.charAt(testStr.length() - i - 1));
```

17. Find the Longest String Element in a List

```
Optional<String> longestString = strings.stream()
    .max(Comparator.comparingInt(String::length));
```

Section 3: Filtering, Collections & Grouping Variations

18. Extract Duplicate Elements From a Collection

Identifies duplicated components by checking if elements fail to insert into a temporary tracking set tracker.

```
List<Integer> duplicateList = Arrays.asList(1, 2, 3, 2, 4, 3, 5, 6, 1);
Set<Integer> uniqueSeen = new HashSet<>();
```

```
Set<Integer> duplicatesFound = duplicateList.stream()
    .filter(i -> !uniqueSeen.add(i))
    .collect(Collectors.toSet());
```

19. Square or Cube Each Element in a List

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
List<Integer> squares = numbers.stream().map(i -> i * i).collect(Collectors.toList());
List<Integer> cubes = numbers.stream().map(i -> i * i * i).collect(Collectors.toList());
```

20. Remove Null Values and Retain Unique Items

```
List<String> dirtyList = Arrays.asList("A", null, "B", "A", null, "C");
List<String> cleanedList = dirtyList.stream()
    .filter(Objects::nonNull)
    .distinct()
    .collect(Collectors.toList());
```

21. Frequency of Each Element in an Array / List

```
List<String> itemGroup = Arrays.asList("Apple", "Banana", "Apple", "Orange", "Banana");
Map<String, Long> elementFrequency = itemGroup.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

22. Flatten a Nested List Matrix (`List>`)

Flattens multi-dimensional list matrices down into a clean, unified single-level sequence layout.

```
List<List<Integer>> nestedList = Arrays.asList(
    Arrays.asList(1, 2, 3),
    Arrays.asList(4, 5),
    Arrays.asList(6, 7, 8)
);
```

```
List<Integer> flatList = nestedList.stream()
    .flatMap(Collection::stream)
    .collect(Collectors.toList());
```

23. Join a List of Strings with a Prefix, Suffix, and Delimiter

```
List<String> words = Arrays.asList("Alpha", "Beta", "Gamma");
String formattedString = words.stream()
    .collect(Collectors.joining(", ", "[", "]"));
// Yields value: "[Alpha, Beta, Gamma]"
```

24. Group Employees by Department (Custom Objects)

```
// Assuming an Employee data entity structure with a getDepartment() accessor
Map<String, List<Employee>> employeesByDept = employeeList.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));
```

25. Calculate a Person's Age from Their Date of Birth

Uses modern Java time APIs to calculate precise durations between dates.

```
LocalDate birthDay = LocalDate.of(1994, 6, 18);
LocalDate today = LocalDate.now();
long calculatedAgeInYears = java.time.temporal.ChronoUnit.YEARS.between(birthDay, today);
```